Cátedra: Programación III

Carrera: Licenciatura en Sistemas

Año: 2026 – 1er Cuatrimestre

Trabajo Práctico Nro: 1

P.O.O. en Python

Facultad de Ciencias de la Administración
Universidad Nacional de Entre Ríos

Objetivos

- El Trabajo Práctico N.º: 1 pretende que el alumno:
  - Aprenda a crear clases e instanciarlas.
  - Ponga en práctica los conceptos principales del paradigma orientado a objetos: abstracción, encapsulamiento, herencia y polimorfismo.
  - Comprenda como tratar con excepciones.
  - Practique el procesamiento de archivos CSV.

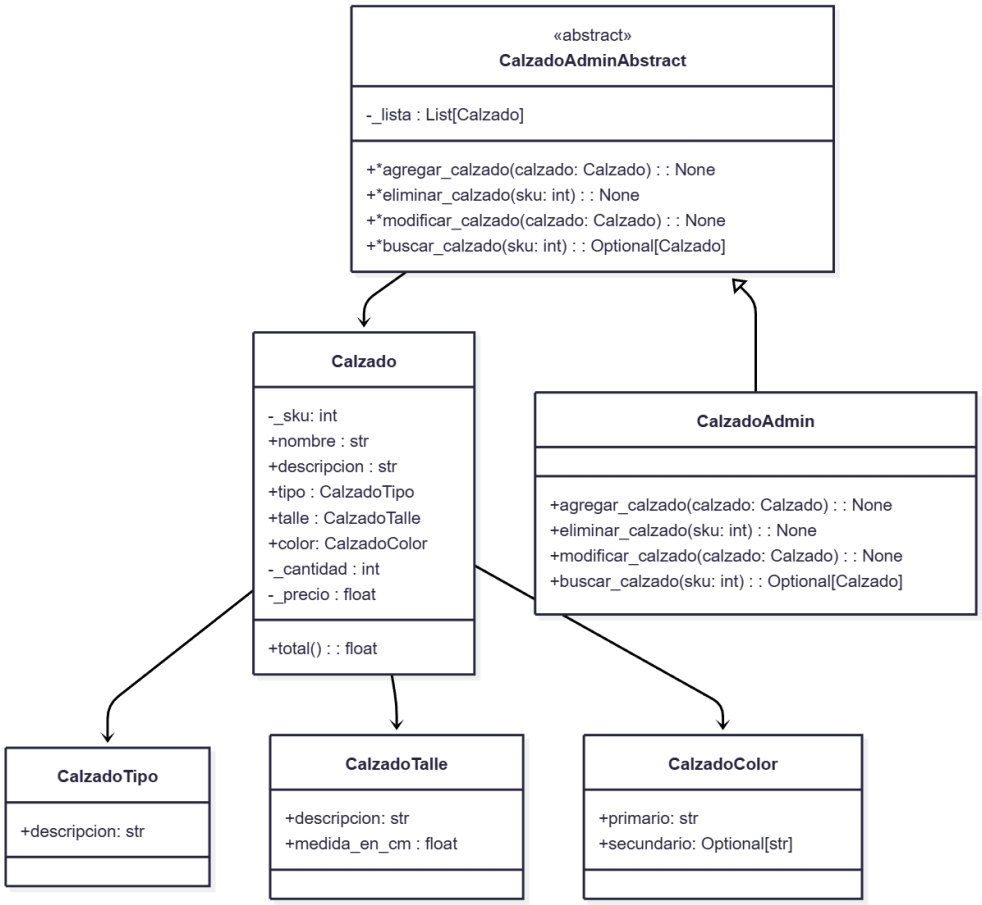
Condiciones de Entrega

- El trabajo práctico deberá ser:
  - Realizado en grupos de no más de 4 (cuatro) alumnos.
  - Cargado en la sección del Campus Virtual correspondiente, junto con los archivos de código fuente que requieran las consignas. Cada uno de ellos en sus respectivos formatos pero comprimidos en un único archivo con formato zip, rar, tar.gz u otro. De manera alternativa puede subirse un archivo txt con el enlace al repositorio GitHub / GitLab con la solución. Si el trabajo práctico fue realizado en grupo, deberá especificarse en el nombre del archivo los apellidos de los integrantes del mismo.
  - Entregado el **Lunes 20 de Abril de 2026**.
- El práctico será evaluado:
  - Con nota numérica y conceptual (Excelente, Muy Bueno, Bueno, Regular y Insuficiente), teniendo en cuenta la exactitud y claridad de las respuestas, los aportes adicionales fuera de los objetivos de los enunciados y la presentación estética.
- Las soluciones del grupo deben ser de autoría propia. Aquellas que se detecten como idénticas entre diferentes grupos serán clasificadas como **MAL** para todos los involucrados en esta situación que será comunicada en la devolución.

- Los ejercicios que exijan codificación se valorarán de acuerdo a su exactitud, prolijidad (identación y otras buenas prácticas).

Ejercicios de Programación

1. El comercio **Rainbow Shoes** encarga a la empresa de desarrollo de software para la que usted trabaja un software que permita llevar control de las existencias sus productos. A tal fin se le solicita la programación de las clases que mantendrán la información en tiempo de ejecución.



Deberá realizar las siguientes tareas:

a) Modifique las clases **`Calzado`**, **`CalzadoTipo`**, **`CalzadoColor`** y **`CalzadoTalle`** implementando:
   I. Constructor parametrizado que inicialice cada una de las variables de instancia con un valor pasado por parámetro.
   II. El método **`__eq__()`**.
   III. El método **`__str__()`**.
   IV. El método **`__repr__()`**.

b) Utilizando **decorators** programe modificadores de acceso para los atributos no públicos.

c) Utilizando **decorators** defina una propiedad sólo lectura de nombre **`total`** que devuelva el producto: **`self._cantidad * self._precio`**.

d) Modifique la clase **`Calzado`** de forma que arroje una excepción de tipo **`ValueError`** cuando:
   I. Se intente establecer un valor negativo para el atributo **`self._sku`**.
   II. Se intente establecer un valor negativo para el atributo **`self._cantidad`**.
   III. Se intente establecer un valor negativo para el atributo **`self._precio`**.

e) Cree el archivo **`calzado_admin.py`** y dentro de él la clase **`CalzadoAdmin`** que implemente **`CalzadoAdminAbstract`** y los método abstractos en ella definidos.

f) Modifique el constructor de **`CalzadoAdmin`** de forma que acepte una lista de elementos de tipo Calzado para inicializar la variable de instancia **`_lista`**. Este parámetro debe ser opcional.

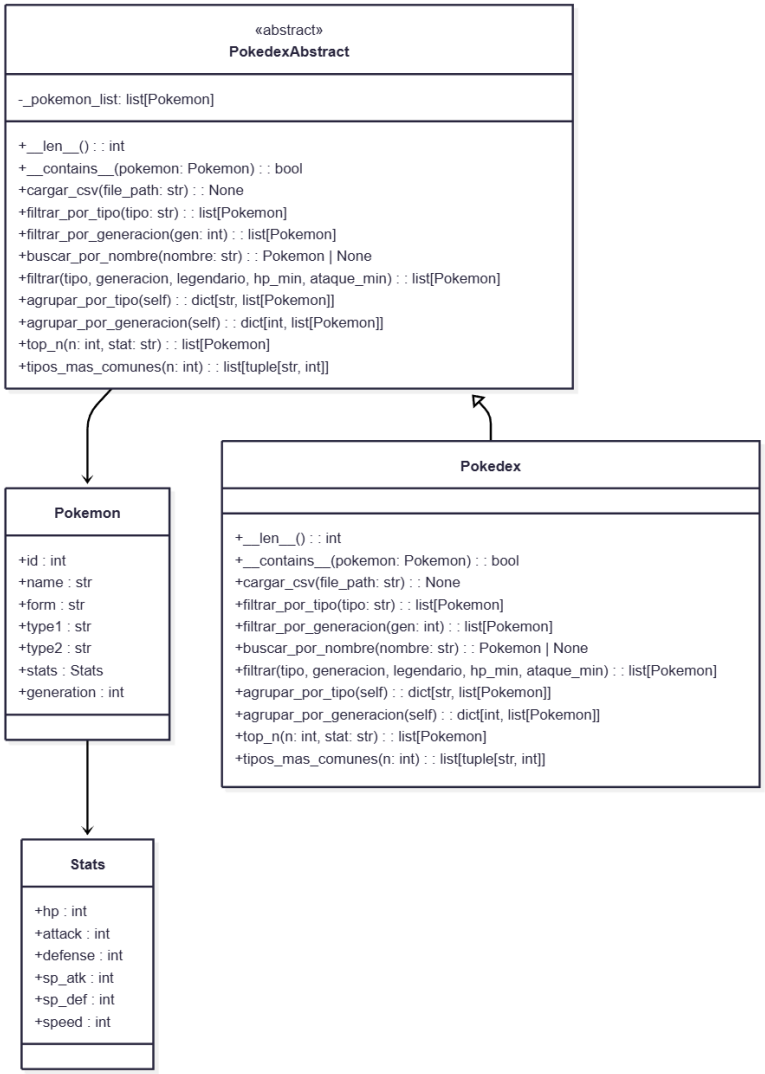
g) Modifique **`CalzadoAdmin`** de forma que arroje una excepción de tipo **`ValueError`** cuando se intente agregar un elemento a la lista de productos que tenga igual **`sku`** que otro ya existente.

h) En el archivo **`main.py`** ponga a prueba los ejercicios anteriores creando al menos 5 instancias de calzado y ponga a prueba los métodos de **`CalzadoAdmin`**.

2. A partir del archivo **`ejercicio2/pokemon.csv`**:

id	name	form	type1	type2	hp	attack	defense	sp_atk	sp_def	speed	generation
1	Bulbasaur		Grass	Poison	45	49	49	65	65	45	1
2	Ivysaur		Grass	Poison	60	62	63	80	80	60	1
3	Venusaur		Grass	Poison	80	82	83	100	100	80	1
4	Charmander		Fire		39	52	43	60	50	65	1
5	Charmeleon		Fire		58	64	58	80	65	80	1

Cumplir con el siguiente diagrama de clases:



a) Crear la clase **`Pokemon`** con los siguientes requisitos:
   I. Constructor parametrizado que inicialice cada una de las variables de instancia con un valor pasado por parámetro.
   II. El método **`__eq__()`**.
   III. Los métodos **`__repr__()`** y **`__str__()`**

IV. Modifique la propiedad **`id`** de forma que sea un atributo sólo lectura.
V. Modifique la propiedad **`generation`** de forma que sea un atributo con acceso protegido por convención (no público) e implemente controles para que no se pueda establecer un valor no entero o negativo.

b) Como clase asociada a **`Pokemon`** y para representar las estadísticas de cada **`Pokemon`** defina la clase **`Stats`**:

I. Usando nombres en snake case, defina los atributos: **`_hp, _attack, _defense, _sp_atk, _sp_def, _speed`**. Todos ellos deberán tener acceso protegido por convención (no público).

II. Agregue controles para que no se puedan establecer valores negativos para estas propiedades. Prevenga esta situación arrojando excepciones del tipo **`ValueError`**.

III. Programe el método **`total_stats()`** que retorne la suma de **`HP + Attack + Defense + Sp. Atk + Sp. Def + Speed.`**

c) Programe la clase **`Pokedex`** como clase derivada de **`PokedexAbstract.`**

d) En un archivo **`main.py`** ponga a prueba **TODOS** los métodos de **`Pokedex`** luego de leer los datos del archivo CSV utilizando el método **`cargar_csv()`** de esa misma clase.

Rúbricas de evaluación

- **Insuficiente: 0%**
- **Regular: 40%**
- **Bueno: 60%**
- **Muy bueno: 80%**
- **Excelente: 100%**

Puntos / calificación por inciso

- **1.a. Total: 10 puntos.**
  - Excelente: Constructor correcto en todas las clases y métodos dunder bien implementados y coherentes.
  - Muy Bueno: Todo implementado pero con detalles menores.
  - Bueno: Faltan mejoras en alguno de los dunder.
  - Regular: Implementación incompleta o incorrecta.
  - Insuficiente: No implementado o programa no funcional.
- **1.b. Total 15 puntos.**
  - Excelente: Uso correcto de **`property`** y **`setters`** en los tres atributos.
  - Muy Bueno: Pequeños errores de implementación.
  - Bueno: Propiedades parcialmente implementadas.
  - Regular: Uso incorrecto o acceso directo.
  - Insuficiente: No implementado.
- **1.c. Total 10 puntos**.
  - Excelente: Property **`total`** correctamente implementada como solo lectura.
  - Muy Bueno: Funciona pero con detalles menores.
  - Bueno: Implementación funcional pero poco clara.
  - Regular: Implementación incorrecta.
  - Insuficiente: No implementado.
- **1.d. Total 20 puntos.**
  - Excelente: Todas las validaciones implementadas correctamente.
  - Muy Bueno: Falta una validación.
  - Bueno: Validación parcial.
  - Regular: Validaciones incorrectas.
  - Insuficiente: No implementado.
- **1.e. Total 70 puntos.**
  - Excelente: Herencia y métodos abstractos correctamente implementados.
  - Muy Bueno: Pequeños detalles.
  - Bueno: Implementación funcional pero incompleta.
  - Regular: Problemas de herencia o implementación.
  - Insuficiente: No implementado.
- **1.f. Total 20 puntos.**
  - Excelente: Controla correctamente duplicados y lanza excepción.
  - Muy Bueno: Control funcional pero mejorable.
  - Bueno: Control parcial.
  - Regular: Implementación incorrecta.
  - Insuficiente: No implementado.
- **1.g. Total 20 puntos.**

    - Excelente: Prueba todos los métodos con varios casos.
    - Muy Bueno: Pruebas correctas.
    - Bueno: Pruebas básicas.
    - Regular: Pruebas insuficientes.
    - Insuficiente: No hay pruebas.
- **2.a. Total 5 puntos.**
    - Excelente: Implementación completa y correcta.
    - Muy Bueno: Pequeños detalles.
    - Bueno: Implementación funcional.
    - Regular: Implementación incompleta.
    - Insuficiente: No implementado.
- **2.b. Total 15 puntos.**
    - Excelente: Implementación completa y validaciones correctas.
    - Muy Bueno: Detalles menores.
    - Bueno: Funcional pero incompleto.
    - Regular: Validaciones incorrectas.
    - Insuficiente: No implementado.
- **2.c. Total 80 puntos.**
    - Excelente: Implementación completa de métodos abstractos.
    - Muy Bueno: Pequeños detalles.
    - Bueno: Implementación parcial.
    - Regular: Problemas en implementación.
    - Insuficiente: No implementado.
- **2.d. Total 20 puntos.**
    - Excelente: Prueba todos los métodos.
    - Muy Bueno: Pruebas correctas.
    - Bueno: Pruebas básicas.
    - Regular: Pruebas incompletas.
    - Insuficiente: No hay pruebas.

Aclaración: Los incisos vinculados a la implementación de las clases administradoras abstractas y concretas poseen una ponderación mayor debido a que integran múltiples competencias del trabajo práctico: uso de clases abstractas, herencia, manejo de colecciones, validaciones, búsquedas y lógica de negocio.